FOCUSORBIT · BUILD JOURNAL

Full Guide

How we are building FocusOrbit, step by step: what we built, the options we had, and why we chose each approach.

Author
Youssef Gamal

Edition
01 · Foundations

Date
25 September 2026

CONTENTS

What's inside

01	The idea	What FocusOrbit is, in one sentence
02	Where we started	Auditing the first version
03	The roadmap	Eight steps, one lesson each
04	Why the database comes first	Data shapes every screen
05	Choosing the stack	Next.js · PostgreSQL · Drizzle
06	The schema	Six tables, one center
07	Design decisions	Eight rules we chose, and why
08	Migrations	The history of the database
09	Docker	Running Postgres, fixing WSL, testing the rules
10	Project map & commands	Where everything lives
11	Glossary	Every new word, explained
12	What's next	The following chapters

How to read this guide

Each chapter follows the same shape: **the problem → the options → what we chose → why**. The code will change; the reasoning is the part worth keeping. Every chapter here also has a short Markdown version in `docs/journal/`.

CHAPTER 01

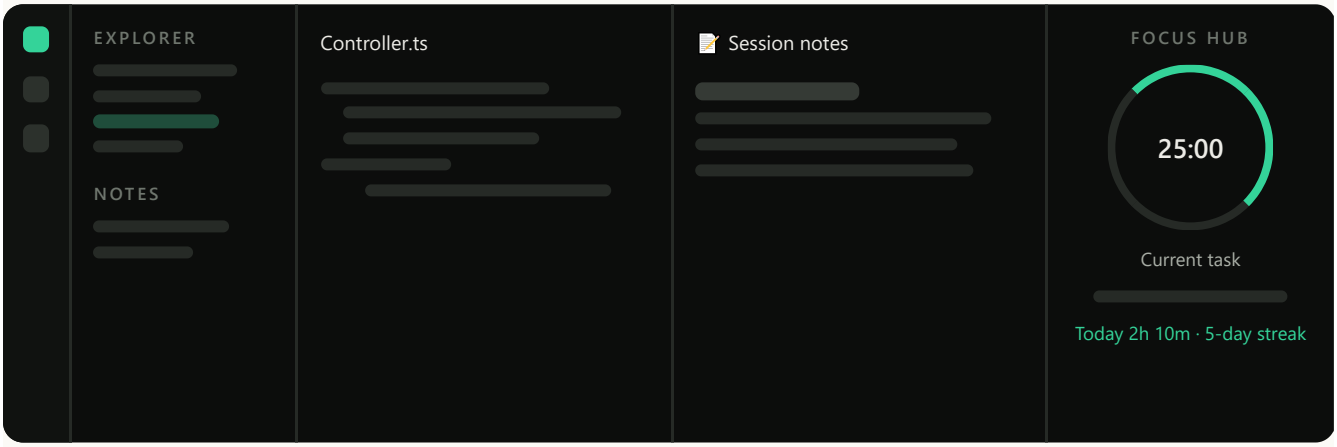
The idea

Before writing any code, we reduced the product to one sentence. Every later decision is checked against it.

A focus session where your code and your notes live side by side, a timer keeps you honest, and everything you did is saved into a history you can look back on.

From four apps to one workspace

The first version had four separate screens: a dashboard, an IDE, a notes app and a landing page. Each one looked good on its own but knew nothing about the others. The new direction is **one workspace**:



Left: files, notes and history. Center: a split editor where a note is just another tab. Right: the focus hub.

The key insight

A note is just another tab

Code and notes share one tab system, so "notes next to the IDE" becomes a split editor instead of a second app.

The session is the center

The timer, notes, code and history all attach to a *focus session*. History is a list of sessions, and motivation stats are calculated from that list.

CHAPTER 02

Where we started

The first version was a well-designed shell with no logic behind it. Before building anything new, we read the code and listed what was actually wrong.

FINDING	WHY IT HAPPENS	LESSON
Two layouts stacked Double sidebar, clipped "Deep Focus" label	`IdeWorkspace.tsx` renders its own fixed header and sidebar, and `workspace/layout.tsx` already renders `Sidebar` and `TopNav`.	Only one component should own the page shell.
The timer dies when you switch views	The `setInterval` lives inside the IDE component. When the view changes, the component unmounts and the interval is cleared.	Long-lived state must not live in a component that can disappear.
The timer drifts	It subtracts 1 every second, but browsers slow down timers in background tabs, so 25 minutes can become 27 or more.	Store `endsAt` and calculate `remaining = endsAt - now`.
All data is fake	The tasks, the "72%", the "42h" and the file tree are hardcoded.	Decide what the app remembers before you build more screens.

The pattern behind three of the four bugs

Each one comes from state living in the wrong place. The fix is the same every time: pick **one source of truth**, and calculate everything else from it.

CHAPTER 03

The roadmap

Eight steps. Each one is small enough to finish and teaches one idea.

#	STEP	WHAT YOU LEARN	STATUS
1	**Database & schema**	Designing the data first; relational modelling; constraints	DONE
1b	**Docker for Postgres**	Images, containers, ports, volumes	DONE
2	A correct timer: `endsAt`, outside components	Source of truth vs. derived state; component lifecycle	NEXT
3	Server actions: start / pause / resume / finish a session	Server vs. client code; data mutations; auth checks	PLANNED
4	One layout shell	Layout composition; removing duplicate UI	PLANNED
5	Notes as editor tabs, with a split view	One abstraction for different content types	PLANNED
6	History view & stats	Calculating numbers from data (SQL aggregation)	PLANNED
7	Design tokens & the premium visual pass	Design systems	PLANNED

The original plan started with the timer. We changed the order to **database first**. The next chapter explains why that was the better call.

CHAPTER 04

Why the database comes first

A screen is a view of some data. If you don't know the data, every screen is a guess.

UI first (what we had)

Screens are designed with placeholder numbers. Later the real data turns out to have a different shape, and the screens have to be rebuilt.

Data first (what we chose)

Decide what the app remembers. Screens then become ways to show and change that data, and fake numbers are replaced by queries.

How we found the tables: nouns in the sentence

Take the one-sentence idea and underline the nouns. Each important noun becomes a table:

IN THE IDEA	TABLE	NOTE
you	`users`	Identity comes from Clerk
what you studied	`topics`	The label that ties everything together
what you planned	`tasks`	Optional link to a topic
a timed block of focus	`focus_sessions`	**The center**
what you wrote	`notes`	Can be written during a session
the code you wrote	`files`	A folder tree
history, motivation	*none*	Calculated from sessions, not stored

CHAPTER 05

Choosing the stack

Three decisions: where the backend lives, which database to use, and how the code talks to it.

Where the backend lives

OPTIONS — **A.** A separate ASP.NET Core API with Clean Architecture (the same structure as the Hrm project) · **B.** Inside Next.js · **C.** Design the schema only, and decide later

CHOSE — **B: inside Next.js.**

WHY — One repo, one language, and the fastest path to a working product. Next.js server actions let the frontend call database code directly. The schema is plain SQL tables, so moving to a .NET API later would mean rewriting the data-access layer, not redesigning the data.

Which database

OPTIONS — PostgreSQL · SQLite · MongoDB

CHOSE — **PostgreSQL.**

WHY — The data is *relational*: sessions belong to topics, notes belong to sessions. Postgres enforces rules with foreign keys, check constraints and partial indexes, and it's what most production apps use. MongoDB would push those rules into our own code. SQLite is great but harder to share between processes and deployments.

How the code talks to it

OPTIONS — Raw SQL · Prisma · Drizzle ORM

CHOSE — **Drizzle ORM** with the `postgres` driver.

WHY — The schema is ordinary TypeScript that reads almost like SQL. There's no separate code-generation step, the types come straight from the schema, and the migrations it writes are plain `.sql` files you can read. You learn SQL instead of hiding from it.

Where Postgres runs

OPTIONS — Docker · Native Windows install · Hosted (Neon/Supabase)

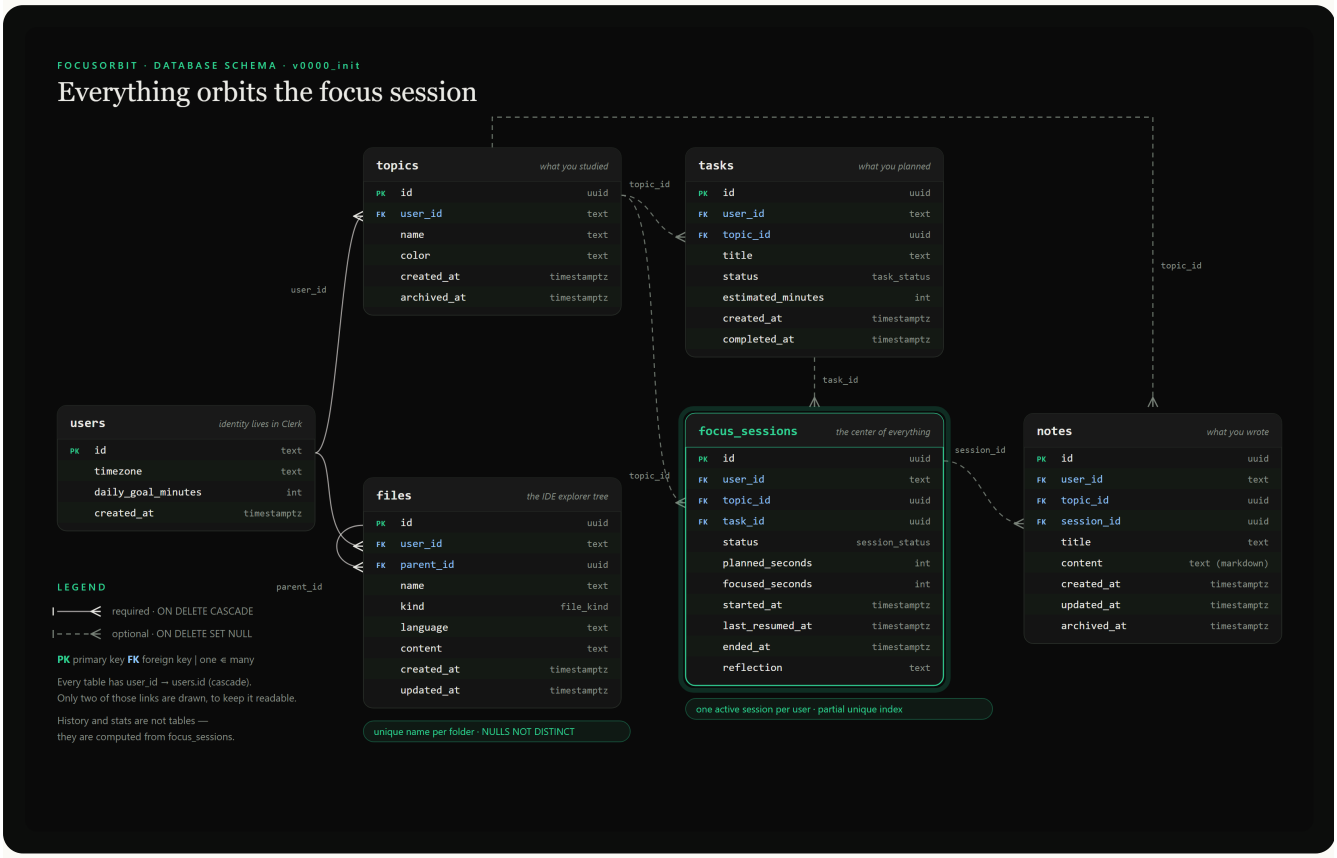
CHOSE — **Docker.**

WHY — One file (`docker-compose.yml`) describes the exact Postgres version and settings. It's the same on every machine, easy to reset, and a skill you'll use in every backend job. See chapter 09.

CHAPTER 06

The schema

Six tables. Everything else is calculated from them.



Solid lines are required and cascade on delete. Dashed lines are optional and become NULL when the parent is deleted. The file `docs/schema.png` is generated by `docs/guide/schema-diagram.mjs`.

TABLE	PURPOSE	NOTABLE COLUMNS
`users`	Your preferences. The `id` is the Clerk user id.	`timezone`, `daily_goal_minutes`
`topics`	What you studied (e.g. "Clean Architecture").	unique `(user_id, name)`
`tasks`	What you planned to do.	`status`: todo / in_progress / done
`focus_sessions`	One timed block of focus. The center of the schema.	`focused_seconds`, `last_resumed_at`, `reflection`
`notes`	Markdown notes, optionally linked to a session and a topic.	`content`, `archived_at`
`files`	The IDE's folder tree.	`parent_id` → `files.id`

CHAPTER 07

Design decisions

The rules we chose, with the reason for each.

1 · Clerk owns identity

RULE `users.id` is the Clerk user id. We don't copy the email, name or avatar.

WHY A copy of someone else's data goes stale. Store only what *your* features need.

2 · Don't store what you can calculate

RULE No `streak`, `total_hours` or `progress` columns. They are queries over `focus_sessions`.

WHY A stored number can disagree with the data it came from; a calculated one can't. Get it correct first, and add a cache later if it's ever slow.

3 · Pause and resume with two columns

RULE No separate pauses table. `focused_seconds` + `last_resumed_at` fully describe a pausable timer.

WHY It's the smallest model that is always correct, and it fixes the drift bug, because the truth is timestamps, not a counter.

```
// resume
last_resumed_at = now
// pause
focused_seconds += now - last_resumed_at ;  last_resumed_at = null
// live focused time, at any moment
focused_seconds + (now - last_resumed_at)
```

4 · The database enforces the rules

RULE Important rules are constraints in Postgres, not just checks in the UI.

WHY Two browser tabs, a double-click, or a future bug can all bypass UI checks. The database is the last line of defense.

RULE	HOW POSTGRES ENFORCES IT
One active session per user	Partial unique index on `user_id` `WHERE status IN ('running','paused')`
Planned time is positive	`CHECK (planned_seconds > 0)`
Folders have no content	`CHECK (kind = 'file' OR content IS NULL)`
No duplicate names in a folder, including the root	`UNIQUE NULLS NOT DISTINCT (user_id, parent_id, name)`

5 · Cascade vs. set null

RULE — **Cascade** when the child can't exist without the parent (a user's data, a folder's contents). **Set null** when the child still has value on its own (delete a topic, keep your notes).

WHY — Deleting is a product decision. Ask, for each link: "if the parent goes, should this go too?"

6 · The file tree is an adjacency list

RULE — Each file row points at its parent folder.

WHY — It's the simplest tree model: moving a file is one `UPDATE`. Reading a whole tree needs `WITH RECURSIVE`, which we'll learn when we build the explorer.

7 · Time is stored in UTC

RULE — Every timestamp is `timestamptz`. The user's timezone is used only to decide what "today" means.

WHY — A study session at 23:30 in Cairo must count towards the right day's streak.

8 · Leave things out on purpose (YAGNI)

RULE — For now there are no break sessions, no tags and no session↔file links.

WHY — Every table is something to maintain. Add one when a real feature needs it; a migration makes that easy.

CHAPTER 08

Migrations

A migration is a numbered SQL file that moves the database from one version to the next. Migrations are to the database what commits are to code.

The loop

```
# 1. change the TypeScript schema
src/db/schema.ts
# 2. Drizzle compares it with the last snapshot and writes the SQL
npm run db:generate -- --name add_something   → drizzle/0001_add_something.sql
# 3. READ the SQL. Never apply a migration you haven't read.
# 4. apply it
npm run db:migrate
```

A piece of our first migration

```
CREATE UNIQUE INDEX "focus_sessions_one_active_per_user"
  ON "focus_sessions" USING btree ("user_id")
  WHERE "focus_sessions"."status" in ('running', 'paused');

CONSTRAINT "files_sibling_name_unique"
  UNIQUE NULLS NOT DISTINCT("user_id","parent_id","name")
```

Why migration files are committed to git

Anyone who clones the repo, including a production server, can rebuild exactly the same database by running the files in order.

CHAPTER 09

Docker

Docker runs Postgres in an isolated box that is described by one file, can be recreated at any time, and is the same on every machine.

Image

A read-only template. Ours is `postgres:17-alpine` : Postgres 17 on a tiny Linux.

Container

A running instance of an image. Ours is named `focusorbit-db` . It's disposable.

Port mapping

`5433:5432` means your PC's port 5433 leads to Postgres's 5432 inside the container. We use 5433 so it can't clash with a native Postgres.

Volume

Storage that outlives the container. `pgdata` holds your tables. `down` keeps it; `down -v` deletes it.

```
services:
  db:
    image: postgres:17-alpine
    environment: { POSTGRES_USER: focusorbit, POSTGRES_PASSWORD: focusorbit, POSTGRES_DB:
focusorbit }
    ports: [ "5433:5432" ]                # host:container
    volumes: [ pgdata:/var/lib/postgresql/data ]   # data survives restarts
    healthcheck: { test: pg_isready ... }       # "is Postgres ready yet?"
```

The first real bug: reading the logs

Docker Desktop was installed, but `docker run hello-world` failed with "*Docker Desktop is unable to start*". Instead of guessing, we read Docker's own log file:

```
com.docker.backend … dockerAPI: stopped (API proxy is serving requests (no virtualization available))
```

CHECK	RESULT	MEANING
Virtualization in the BIOS	Enabled ✓	The hardware is fine
`wsl` command	Printed only the old usage text	The **WSL 2** feature isn't installed
Docker engine	Not running	It needs WSL 2 to run Linux containers

The fix, step by step

#	ACTION	WHAT WE SAW
1	Enable the Windows features (admin): `dism … Microsoft-Windows-Subsystem-Linux` `dism … VirtualMachinePlatform`	Exit code `3010` : success, *restart required*
2	Restart with Start → Power → **Restart** (on Windows 10, "Shut down" uses Fast Startup and may not apply new features)	`HypervisorPresent` changed from False to **True**
3	`wsl --update --web-download` (admin)	The built-in `wsl.exe` was replaced by **WSL 2.7.14**
4	Start Docker Desktop	Engine up in 15 s · `hello-world` ✓
5	`db:up` → `db:migrate` → `db:seed`	Healthy in 8 s · 6 tables · demo data ✓

A mistake worth keeping: we first tried `wsl --install --web-download` , but that option belongs to `--update` . Reading the tool's own help text showed the right command.

Proving the rules hold

A rule you haven't tried to break is a rule you haven't tested. We attacked the database on purpose:

ATTEMPT	RESULT
A second *running* session for the same user	❌ `focus_sessions_one_active_per_user`
`planned_seconds = 0`	❌ `focus_sessions_planned_positive`
A duplicate `src` folder at the root	❌ `files_sibling_name_unique` (NULLS NOT DISTINCT)
A folder with content	❌ `files_folder_has_no_content`

Lessons: when a tool fails with a vague message, find its log, because the real reason is usually written down. And when the docs and your memory disagree, trust `--help` .

CHAPTER 10

Project map & commands

FILE	WHAT IT DOES
`src/db/schema.ts`	The tables, written in TypeScript. The source of truth for the database.
`src/db/index.ts`	The database connection. Marked `server-only` so it can never reach the browser, and cached on `globalThis` so hot reload doesn't open a new pool on every save.
`src/db/seed.ts`	Creates a demo user with topics, a task, a session, a note and files.
`drizzle/0000_init.sql`	The first migration: the SQL that builds everything.
`drizzle.config.ts`	Tells drizzle-kit where the schema and the database are. It loads `.env.local` through `@next/env`, as the Next.js docs recommend.
`docker-compose.yml`	The Postgres container.
`.env.local` / `.env.example`	The database URL. `.env.local` is gitignored; the example is committed.
`docs/journal/`	Short Markdown entries, one per step.
`docs/guide/`	The source of this PDF and the schema diagram.

```
npm run db:up        # start Postgres (Docker)
npm run db:migrate   # apply migrations
npm run db:seed      # demo data
npm run db:studio    # browse tables in the browser
npm run db:generate  # write a new migration after a schema change
npm run db:down      # stop Postgres (data is kept)
```

CHAPTER 11

Glossary

Primary key (PK)	The column that uniquely identifies a row, e.g. `id`.
Foreign key (FK)	A column that points to a row in another table. Postgres refuses values that point to nothing.
Constraint	A rule the database enforces on every write: `NOT NULL`, `UNIQUE`, `CHECK`, FK.
Index	A lookup structure that makes queries fast. A *unique* index also forbids duplicates.
Partial index	An index over only the rows that match a `WHERE` clause.
Enum	A type with a fixed list of values, e.g. `session_status`.
Cascade / Set null	What happens to child rows when their parent is deleted.
Adjacency list	A tree stored as rows that each point to their parent.
timestamptz	A moment in time, stored in UTC and converted for display.
ORM	A library that maps tables to code. Ours is Drizzle.
Migration	A versioned SQL file that changes the database schema.
Seed	A script that fills the database with sample data.
Derived state	A value calculated from other data instead of stored on its own.
YAGNI	"You Aren't Gonna Need It": don't build a feature before it's needed.
Image / Container / Volume	Docker's template, running instance, and persistent storage.
WSL 2	Windows Subsystem for Linux, the Linux kernel Docker Desktop runs on.

CHAPTER 12

What's next

This guide will grow one chapter at a time.

DONE

Database is live

Postgres runs in Docker, the migration is applied, the demo data is seeded, and all four rules were tested.

NEXT

The session engine

Server actions: start, pause, resume and finish a session, each checking the Clerk user.

THEN

A correct timer

A client timer driven by `endsAt` and synced with the database, so it never drifts.

Design the data. Enforce the rules where they can't be bypassed. Calculate what you can. Write down why.
